

微程序控制的运算器设计

班级：4

姓名：陈俊宏

学号：202300130234

完成日期：2025 年 11 月 28 日

二、目录

1、指令具体设计

2、总体结构设计

3、控制部件设计

A. 设计图介绍

B. 微指令执行流程

C. 微指令设计

D. 指令应用与实现

4、实验总结

三、正文

1、具体指令设计：

MOV R0,#0x55AA

将立即数放置 R0（直接寻址）

LDR R1,0xA0

将 A0 地址处的内容放置 R1（寄存器寻址）

ADD R0,R1

R1+R0->R0

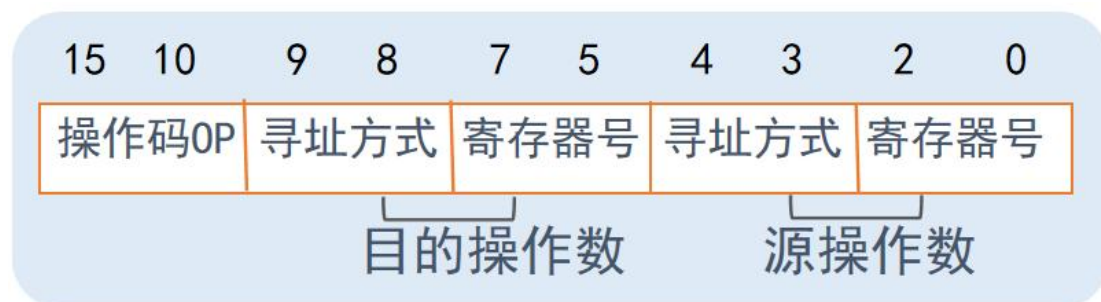
还有 AND、OR、XOR、SUB 等逻辑、算术运算，把 ALU 器件上实现的基本功能都加载进指令

STR R0,[R2]

把 R0 内容放置在 R2 所存地址中(寄存器间接寻址)

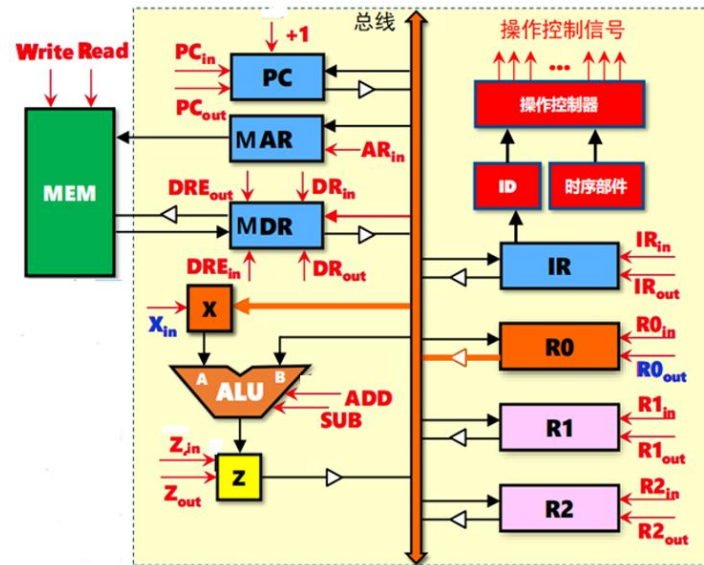
HALT

停机



*操作码格式

2、总体结构设计：

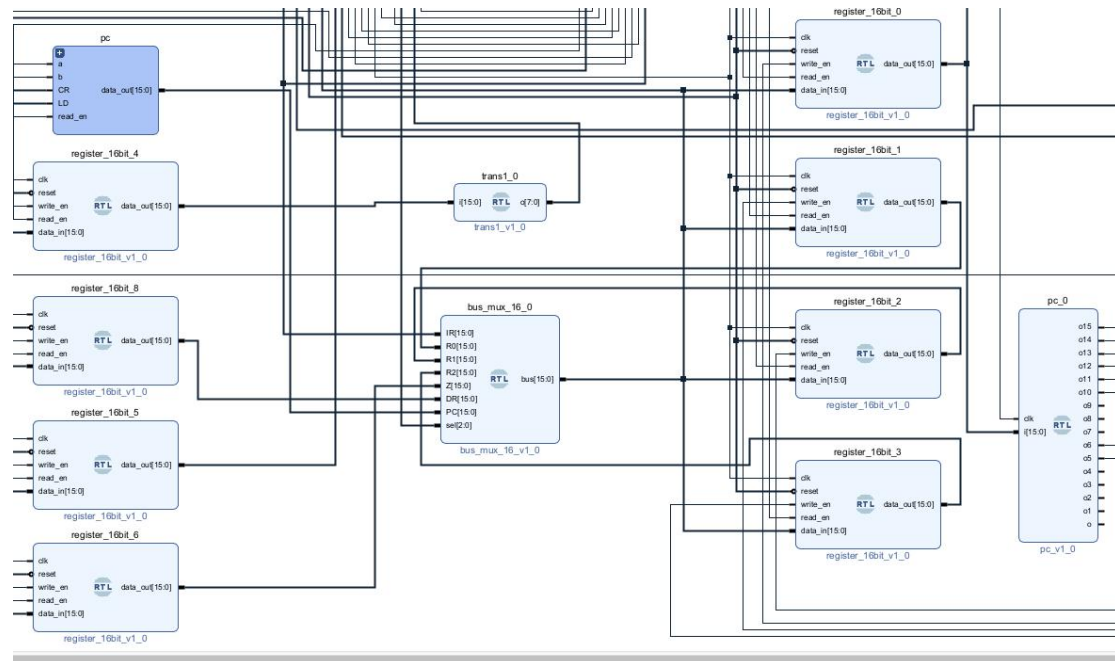


IR: 指令寄存器，存放 pc 指令

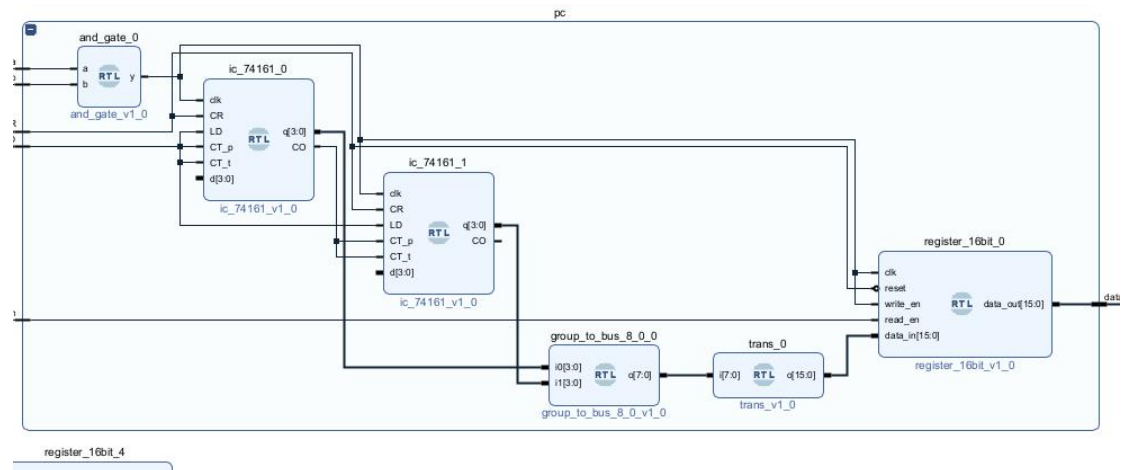
R0、1、2: 普通寄存器，存放普通数据、地址

MAR: 与内存互通的地址寄存器，按该寄存器的地址拿出对应数据

MDR: 保存内存的数据以实现 cpu 与内存数据的互通



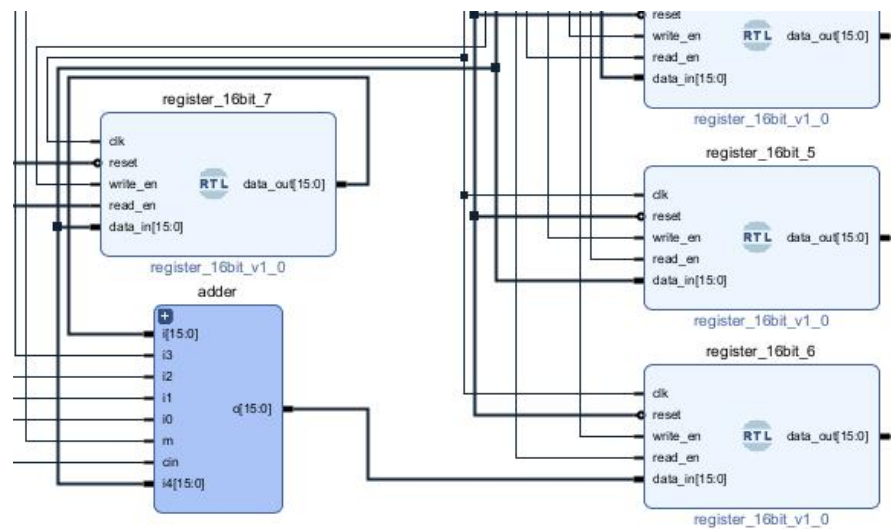
PC: 计算、存储指令地址



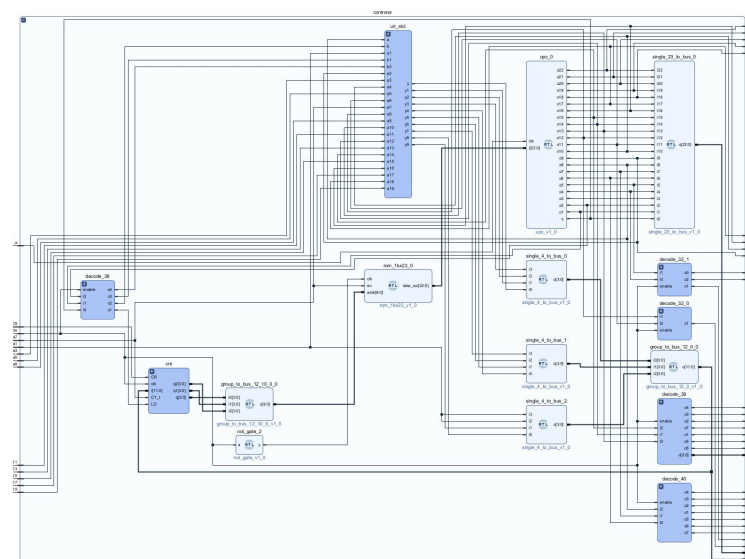
X: 运算器的参数寄存器

ALU: 运算器，实现逻辑、算术运算

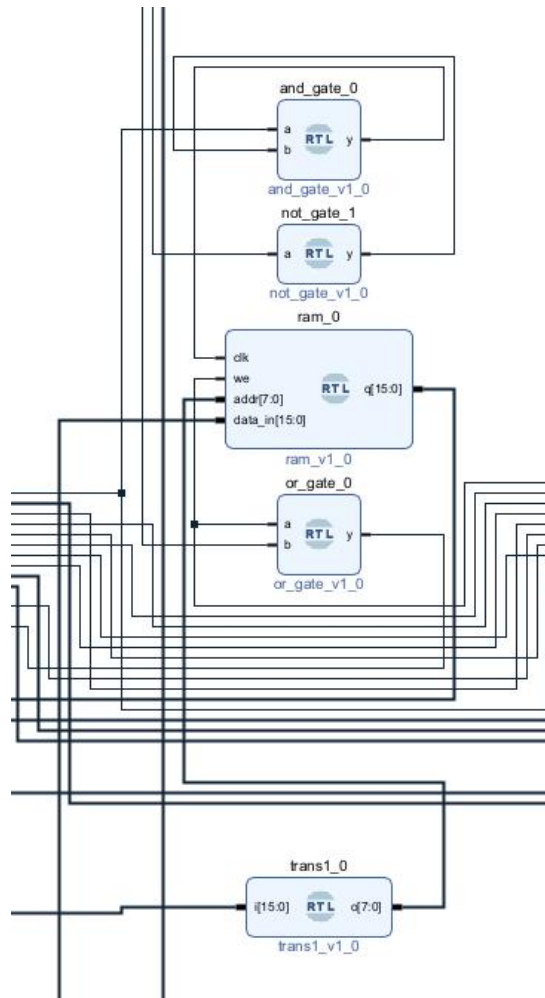
Z: 结果寄存器，存放运算结果



CU: 控制部件，管理微指令、统配硬件执行

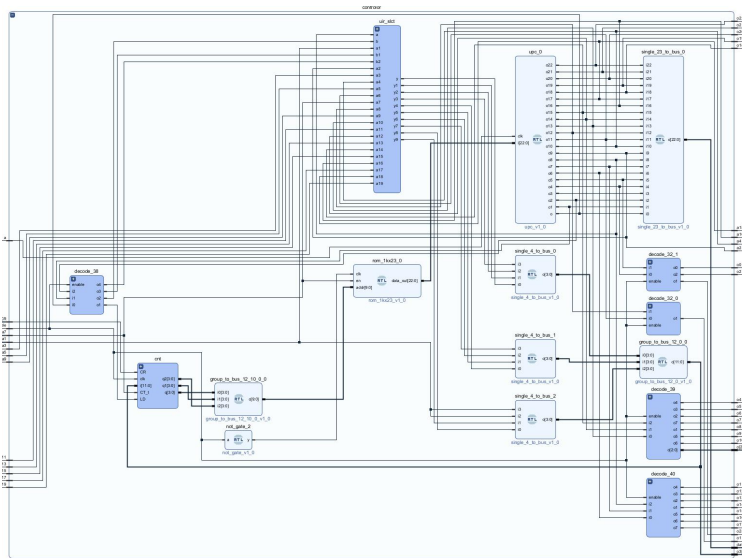


MEM: 内存，存放数据

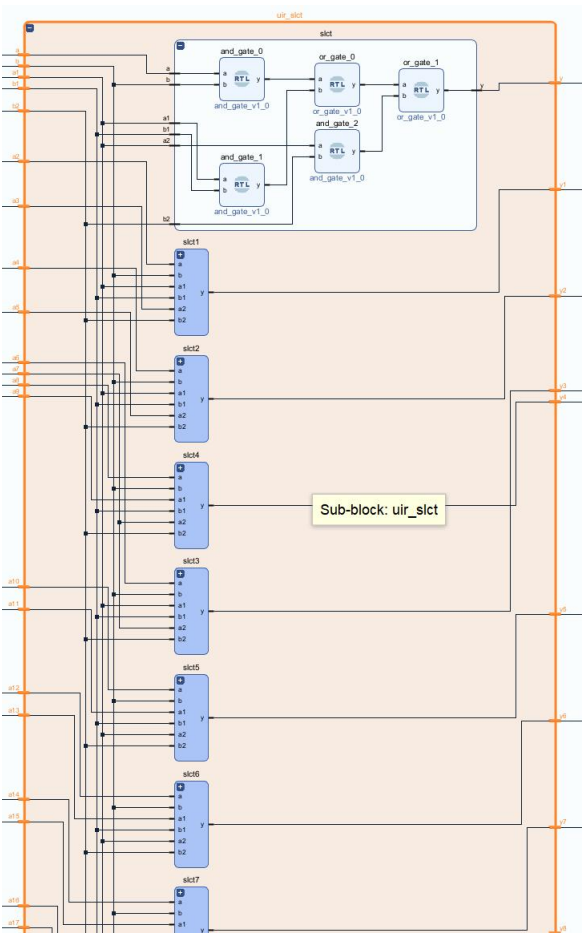


3、控制部件设计：

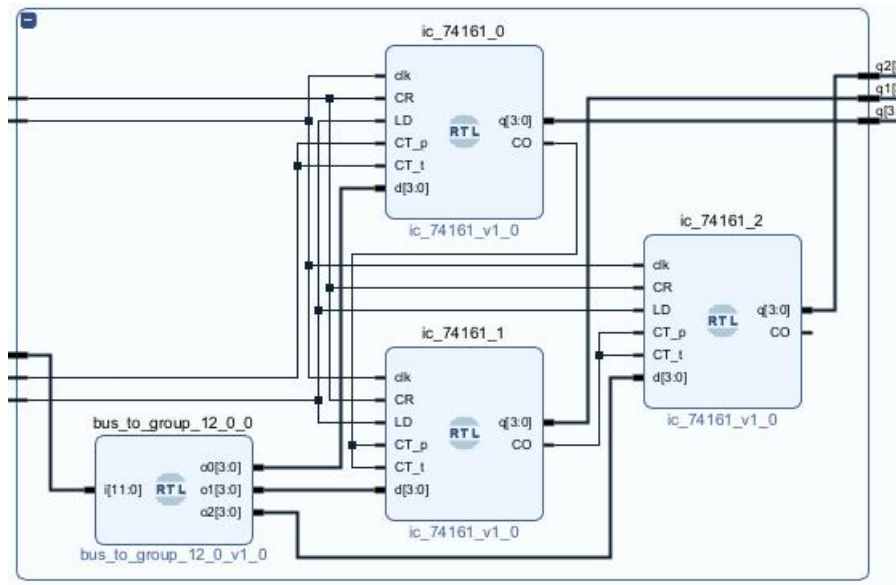
A. 设计图介绍



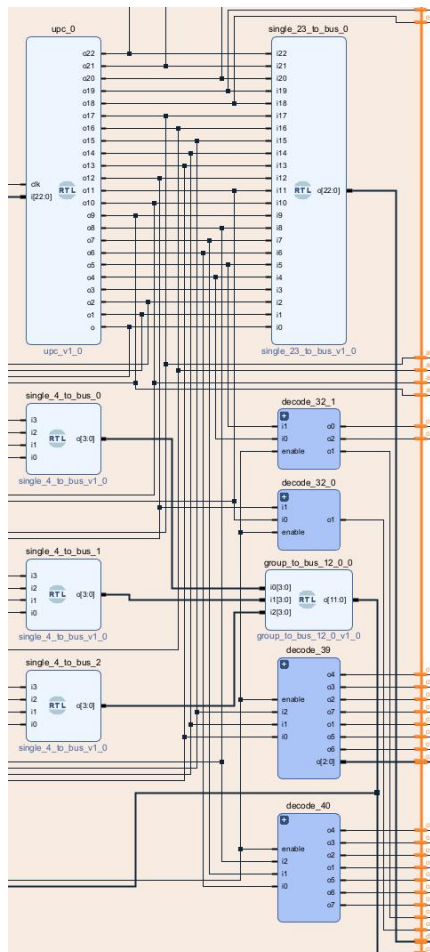
微地址译码电路：



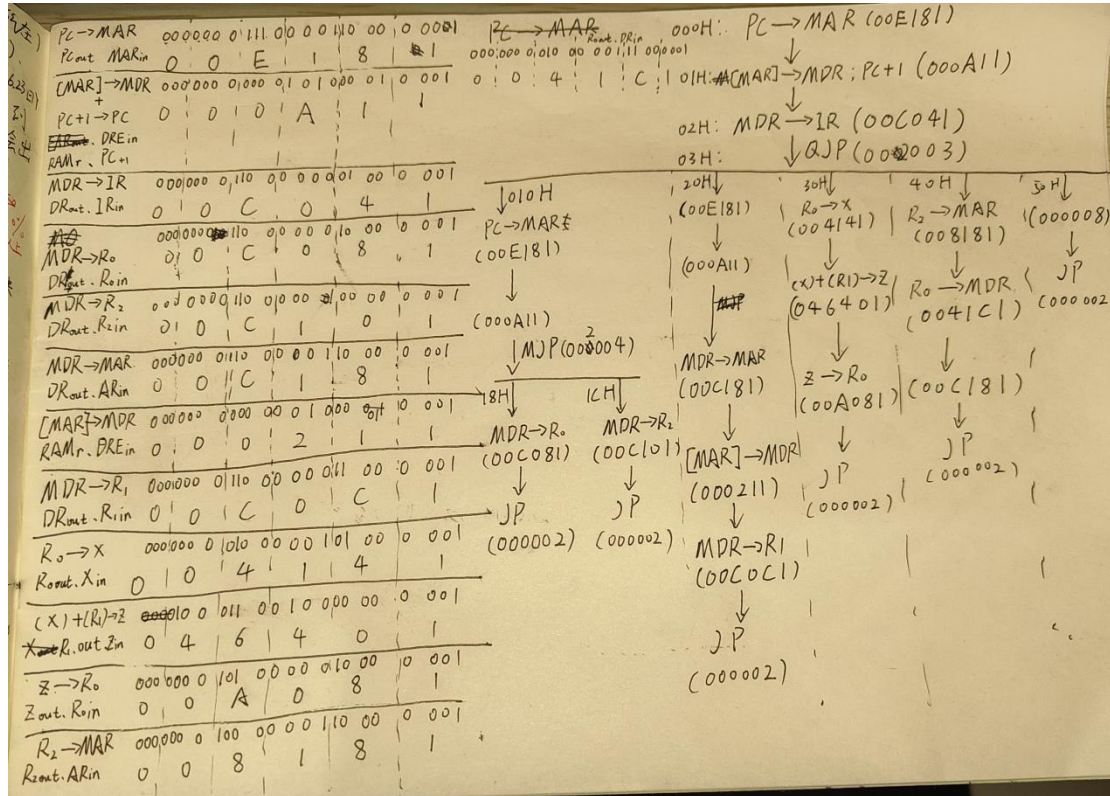
时序控制电路：



微指令译码电路：



B. 微指令执行流程



C. 微指令设计

微指令设计可见上图，具体地址安排在附件 rom.mem 中

D. 指令应用与实现

```
MOV R0,#0x55AA
0x0500,0x55AA
```

```
LDR R1,0xA0
0x0938,0x00A0
```

```
ADD R0,R1
0x0D09
```

```
MOV R2,#0x00FA
0x0540,0x00FA
```

```
STR R0,[R2]
0x1112
```

```
HALT
0x1400
```

4、实验总结：

本此实验实现了微指令微型机的基本功能，该机械体系可分为两个区域：微指令控制区以及功能硬件实现区。

微指令控制区通过计算输出功能硬件的微操作，统一调配每一个时序的运作硬件，从而实现数据的计算、传输；功能硬件则通过接收微指令的控制，完成 in、out、计算功能选择等操作，进而实现实际数据的处理。

但上述体系的构建需要硬件的提前规划和合理搭配，否则只能实现示例中有限的功能。首先，由于 rom 地址跳转的限制，每一个指令只有 16 个微指令操作，且由于该地址空间大小的限制，每个指令中涉及到寄存器的跳转都非常受限，故微指令地址跳转只涉及目的寄存器，源寄存器对于该体系来说几乎没意义。为了进一步使该微型机通用化，可以把微指令的地址跳转扩大几位而不只是每 16 一个指令，这样就能为指令的微操作腾出更多空间，从而实现通用性；其次对于微指令 jp 的地址跳转规划得很不合理，它跳转的地址是 uIR8~uIR17，其中 uIR16 是 DREout、uIR15~13 是寄存器的 out，但是 uIR10 是 Zin、uIR9 是 DREin、uIR8 是寄存器 in 的高位，一旦 out 和 in 同时被触发，这就意味着在微指令运行 JP 功能时，数据是可能会被改动的，而且 uIR11 和 12 是控制 pc 的微指令，若跳转的地址中它们两位是 01，则会让 pc 自动跳转下一位，从而导致程序崩溃。

那么为什么示例实验没有出现上述问题呢？因为它太过简单。

1、尽管每条指令只有 16 个微指令空间，它也能实现基本的微型机功能，它对于基本功能是足够的，只不过它是受限的、没有通用性。

2、对于 JP 微指令无条件跳转，它在示例中不需有很复杂的 JP 跳转，只需跳回 000H 地址重新执行取指就行，而 000H 作为全零刚好又规避了上述的问题，因为全零对于微指令设计都代表无效。

因此，该微型机在此基础上能有很大扩展空间，最终能基本实现现代计算机的功能。